

Análisis Teórico y Experimental de Algoritmos de Divide y Vencerás: Ordenamiento, Búsqueda y Selección

Andres Barbosa[†], Emmanuel Velásquez[†] y Diego Oyarzo[†]

[†]Universidad de Magallanes

Este informe fue compilado el 15 de mayo de 2026

Resumen

El presente proyecto analiza empírica y teóricamente el comportamiento de algoritmos basados en el paradigma de Divide y Vencerás, aplicándolos sobre un robusto sistema de gestión de deportistas construido nativamente en C. El estudio abarca algoritmos de ordenamiento como Merge Sort (en su vertiente clásica y optimizada mediante Insertion Sort) y Quick Sort utilizando el esquema particional de Lomuto con 4 heurísticas de pivote. Asimismo, se incorporan esquemas de Búsqueda Binaria (recursiva y delimitación de rangos), Búsqueda Exponencial y Búsqueda por Interpolación; cerrando el abanico con Quick Select para la búsqueda del k-ésimo elemento.

Mediante evaluaciones sobre variados tamaños de entrada y arreglos dispares, los resultados arrojan evidencia rigurosa sobre el salto cualitativo en la complejidad temporal asintótica (de $\mathcal{O}(n^2)$ a $\mathcal{O}(n \log n)$) en comparación a técnicas sistemáticas previas y explora en profundidad cómo la selección paramétrica del pivote o los umbrales de combinación moldean determinantemente el rendimiento final en escenarios prácticos.

Keywords: *Divide y Vencerás, Merge Sort, Quick Sort, Búsqueda Exponencial, Quick Select, Complejidad Temporal, Lenguaje C, Lomuto*

■ Índice

1	Introducción	3
2	Objetivo principal	3
2.1	Objetivos secundarios	3
3	Marco Teórico	4
3.1	Ordenamiento	4
3.2	Búsqueda	4
3.3	Selección	4
3.4	Criterio de Evaluación	4
4	Desarrollo	5
4.1	Análisis Teórico de Algoritmos de Ordenamiento	5
	Merge Sort: Clásico y Optimizado • Quick Sort y sus Variantes de Pivote	

4.2	Análisis Teórico de Algoritmos de Búsqueda	6
	Búsqueda Binaria: Recursiva y por Rango • Búsqueda Exponencial • Búsqueda por Interpolación	
4.3	Análisis Teórico de Algoritmos de Selección	7
	Quick Select	
4.4	Resultados Experimentales	8
	Ordenamiento: Mejor, Peor y Promedio • Impacto del Umbral en Merge Sort • Desempeño en Algoritmos de Búsqueda • Selección con Quick Select	
4.5	Aplicación Práctica y Funcionalidades	16
	Menú Principal y Opciones Generales • Menú Data Analytics & Rankings • Estructura de Datos y Gestión de Memoria • Aplicación de Paradigmas Algorítmicos • Validación Operativa	
5	Conclusiones	19
6	Contribución por integrante	20
6.1	Andrés Barbosa	20
6.2	Emmanuel Velásquez	20
6.3	Diego Oyarzo	20
7	Anexos	21
7.1	Registro de validación estructural	21
7.2	Código base de partición	21
7.3	Extractos del registro de ejecución	21

1. Introducción

El diseño de algoritmos tiene como objetivo encontrar soluciones a problemas de manera eficiente teniendo en cuenta las limitaciones de tiempo y memoria que existen dentro de los sistemas informáticos.

Tres de los problemas más comunes en la computación son el **ordenamiento** de datos, la **búsqueda** de información y la **selección** de elementos dentro de estructuras de datos; estos problemas se pueden afrontar desde distintas perspectivas, y en este informe los abordamos desde el paradigma de **Divide y Vencerás**.

El presente informe aborda estos problemas sobre una base de datos ficticia de deportistas (generada de manera procedimental) con el fin de exponer, analizar y comparar algoritmos de ordenamiento (*Merge sort* clásico y optimizado, *Quick sort* con múltiples estrategias de pivote), algoritmos de búsqueda (*Búsqueda binaria recursiva*, *exponencial* e *interpolación*) y selección (*Quick select*). Todo ello analizando su comportamiento a nivel teórico y contrastándolo mediante experimentación empírica.

2. Objetivo principal

Analizar y comparar el comportamiento de algoritmos de ordenamiento, búsqueda y selección basados en la estrategia de *Divide y Vencerás*, estableciendo una correspondencia entre los resultados teóricos y experimentales aplicados a un sistema de gestión.

2.1. Objetivos secundarios

1. Comprender el funcionamiento e implementar correctamente algoritmos con estrategia recursiva de partición.
2. Analizar empíricamente la complejidad evaluando el impacto de optimizaciones (ej: umbrales en Merge Sort) y variantes (ej: tipos de pivotes en Quick Sort).
3. Integrar los algoritmos al sistema desarrollado empleando criterios de justificación basados en la eficiencia.
4. Documentar y contrastar de manera gráfica la mejora sustancial en tiempos de ejecución frente a los algoritmos con complejidad asintótica de orden cuadrático de iteraciones previas.

3. Marco Teórico

El presente proyecto se apoya en el paradigma de *Divide y Vencerás*, una estrategia de diseño algorítmico que resuelve un problema grande dividiéndolo en subproblemas más pequeños, resolviendo cada uno de forma recursiva y combinando luego sus resultados. Esta técnica suele producir mejoras sustanciales frente a enfoques iterativos o de exploración completa, especialmente cuando las operaciones de división y combinación son más baratas que procesar la entrada completa de manera secuencial.

3.1. Ordenamiento

Los algoritmos de ordenamiento estudiados en este trabajo incluyen variantes clásicas y optimizadas de *Merge Sort* y *Quick Sort*. *Merge Sort* garantiza complejidad temporal $\mathcal{O}(n \log n)$ en todos los casos, a costa de memoria auxiliar adicional. *Quick Sort*, por su parte, conserva un comportamiento promedio de $\mathcal{O}(n \log n)$, pero depende de la elección del pivote y puede degradarse a $\mathcal{O}(n^2)$ en escenarios desfavorables.

La versión optimizada de *Merge Sort* introduce un umbral para delegar subarreglos pequeños a *Insertion Sort*, reduciendo el costo constante de la recursión y mejorando el rendimiento empírico sin alterar la cota asintótica principal.

3.2. Búsqueda

Para la búsqueda de elementos en arreglos ordenados se emplean técnicas como *Binary Search*, *Exponential Search* e *Interpolation Search*. La búsqueda binaria reduce el espacio de búsqueda a la mitad en cada paso y ofrece complejidad $\mathcal{O}(\log n)$. La búsqueda exponencial permite localizar intervalos rápidamente antes de aplicar una búsqueda binaria, mientras que la interpolación aprovecha la distribución de los datos para estimar la posición probable del elemento.

3.3. Selección

Quick Select es el algoritmo de selección utilizado para hallar el elemento K-ésimo sin ordenar completamente el arreglo. Su comportamiento promedio es lineal, $\mathcal{O}(n)$, ya que descarta en cada iteración la mitad del espacio que no contiene la respuesta. En el peor caso, al igual que *Quick Sort*, puede alcanzar $\mathcal{O}(n^2)$ si el particionamiento es muy desbalanceado.

3.4. Criterio de Evaluación

La comparación experimental del proyecto se basa en medir tiempos de ejecución, observar el efecto de la estructura de los datos y contrastar dichos resultados con el análisis asintótico. De este modo, el marco teórico no solo describe los algoritmos, sino que también justifica por qué ciertas optimizaciones, como el umbral en ordenamiento o la elección del pivote, tienen impacto directo sobre el comportamiento real del sistema.

4. Desarrollo

4.1. Análisis Teórico de Algoritmos de Ordenamiento

En esta sección se detallan los algoritmos de ordenamiento implementados bajo el paradigma de *Divide y Vencerás*. Se analizará su funcionamiento de manera teórica, desglosando sus pseudocódigos y estudiando su complejidad asintótica global.

4.1.1. Merge Sort: Clásico y Optimizado

El algoritmo Merge Sort basa su funcionamiento en dividir el arreglo por la mitad repetidamente hasta obtener sub-arreglos de un solo elemento, para luego fusionarlos (operación *merge*) de manera ordenada.

Algorithm 1 Merge Sort Clásico ▷ $V[beg..end]$

```

1: procedure MERGESORT( $V, beg, end$ )
2:   if  $beg < end$  then
3:      $m \leftarrow beg + \lfloor \frac{end-beg}{2} \rfloor$ 
4:     MERGESORT( $V, beg, m$ )
5:     MERGESORT( $V, m + 1, end$ )
6:     MERGE( $V, beg, m, end$ )

```

Análisis de Complejidad: Merge Sort siempre divide el arreglo en dos mitades, tomando tiempo constante $\mathcal{O}(1)$. La función MERGE combina dos sub-arreglos procesando todos sus n elementos, lo cual toma $\mathcal{O}(n)$. La relación de recurrencia aplicable es:

$$T(n) = 2T(n/2) + \mathcal{O}(n)$$

Por el Teorema Maestro, esto resulta en una complejidad temporal de $\mathcal{O}(n \log n)$ en el mejor, peor y caso promedio.

Variante Optimizada con Umbral (Threshold): Al delegar excesivamente al sistema la creación de árboles recursivos inmensos de llamadas y arreglos subyacentes mínimos, se penaliza el rendimiento. Algoritmos de orden cuadrático como *Insertion Sort* poseen gastos constantes diminutos para entradas limitadas. La variante optimizada introduce un límite de recursión (*UMBRAL* o *threshold*); si el tamaño de un sub-arreglo alcanza un $n \leq \text{Threshold}$, se corta la fragmentación y se despacha a la rutina de Insertion Sort, mejorando notablemente los tiempos empíricos sin alterar su cota asintótica principal $\mathcal{O}(n \log n)$.

4.1.2. Quick Sort y sus Variantes de Pivote

Quick Sort selecciona un elemento referencial o *pivote* y particiona el arreglo utilizando el esquema de partición original de **Lomuto**. Todos los datos menores que el pivote recaen iterativamente a su lado izquierdo, mientras que los remanentes se asientan a su derecha. Finalmente, propaga sus llamadas lógicas separadamente.

Algorithm 2 Quick Sort Recursivo ▷ Basado en partición de Lomuto

```

1: procedure QUICKSORT( $V, low, high$ )
2:   if  $low < high$  then
3:      $p \leftarrow \text{LOMUTOPARTITION}(V, low, high)$ 
4:     QUICKSORT( $V, low, p - 1$ )
5:     QUICKSORT( $V, p + 1, high$ )

```

Dependiendo de cómo se selecciona el pivote antes de particionar, el algoritmo lidia diferentemente con las anomalías de los datos (como conjuntos ya previamente ordenados). Se implementaron varias directrices:

- **Último o Primer elemento:** Determinista e inherentemente veloz, pero susceptible y frágil frente a datos ya estructurados.
- **Elemento Aleatorio:** Selecciona un elemento aleatorio mediante un generador pseudoaleatorio. Es probabilísticamente resiliente frente al *peor caso*.
- **Mediana de tres:** Escoge matemáticamente la mediana entre el primer, el central y el último elemento; aislando de forma consistente las desviaciones extremistas.

Análisis de Complejidad Global:

- **Mejor y Caso Promedio** $\mathcal{O}(n \log n)$: Estructura el árbol recursivo de forma simétrica si el pivote divide permanentemente el arreglo en dos particiones proporcionales.
- **Peor Caso** $\mathcal{O}(n^2)$: Ocurre si el esquema particional selecciona ininterrumpidamente un máximo o mínimo. (ej., Pivote del último elemento en arreglos totalmente ordenados).

4.2. Análisis Teórico de Algoritmos de Búsqueda

4.2.1. Búsqueda Binaria: Recursiva y por Rango

Algorithm 3 Búsqueda Binaria Recursiva $\triangleright V[beg..end]$ ordenado

```

1: procedure BINSEARCHREC( $V, beg, end, x$ )
2:   if  $beg > end$  then
3:     return  $-1$ 
4:    $m \leftarrow beg + \lfloor \frac{end - beg}{2} \rfloor$ 
5:   if  $V[m] == x$  then
6:     return  $m$ 
7:   else if  $V[m] < x$  then
8:     return BINSEARCHREC( $V, m + 1, end, x$ )
9:   else
10:    return BINSEARCHREC( $V, beg, m - 1, x$ )

```

Al igual que su versión iterativa de la tarea pasada, fragmenta la zona objetivo fraccionalmente. Conlleva una complejidad temporal de $\mathcal{O}(\log n)$ para el peor y el caso promedio.

Por otro lado, la variante de **Búsqueda por Rango** aplica este mismo principio logarítmico para dictar umbrales a los topes izquierdo y derecho del arreglo de matches para un solo elemento x , retornando límites masivos usando sólo una superposición asintótica de $\mathcal{O}(\log n)$.

4.2.2. Búsqueda Exponencial

La búsqueda exponencial o *crecimiento galopante* halla una delimitación asumiendo aumentos en potencias de 2 (saltando multiplicativamente índices como 1, 2, 4, 8...), para finalmente ceder la limitación precomputada a una búsqueda binaria tradicional.

Algorithm 4 Búsqueda Exponencial

```

1: procedure EXPSEARCH( $V, n, x$ )
2:   if  $V[0] == x$  then
3:     return 0
4:    $i \leftarrow 1$ 
5:   while  $i < n$  and  $V[i] \leq x$  do
6:      $i \leftarrow i * 2$ 
7:   return BINSEARCH( $V, i/2, \min(i, n - 1), x$ )

```

Posee implicaciones a nivel temporal $\mathcal{O}(\log i)$, donde i es el índice del elemento. Convirtiéndola en una opción especialmente rápida en grandes bases de datos.

4.2.3. Búsqueda por Interpolación

Apoya su funcionalidad interpretando numéricamente el "valor" lógico a buscar intersecándolo en una recta analítica generada entre las fronteras.

Algorithm 5 Búsqueda por Interpolación

```

1: procedure INTERPOLATIONSEARCH( $V, n, x$ )
2:    $low \leftarrow 0, high \leftarrow n - 1$ 
3:   while  $low \leq high$  and  $x \geq V[low]$  and  $x \leq V[high]$  do
4:      $pos \leftarrow low + \lfloor \frac{(x - V[low]) * (high - low)}{(V[high] - V[low])} \rfloor$ 
5:     if  $V[pos] == x$  then
6:       return  $pos$ 
7:     if  $V[pos] < x$  then  $low \leftarrow pos + 1$ 
8:     if  $V[pos] > x$  then  $high \leftarrow pos - 1$ 
9:   return -1

```

En el mejor de los casos (distribuciones de datos absolutamente lineales) su comportamiento salta a proezas de orden minúsculo $\mathcal{O}(\log(\log n))$. Su fragilidad (Peor Caso, donde recae en $\mathcal{O}(n)$) yace en su estricta dependencia de si el conjunto del usuario de sistema expone vacíos gigantes de distribución de puntajes.

4.3. Análisis Teórico de Algoritmos de Selección**4.3.1. Quick Select**

Comparte el mismo árbol generativo de Lomuto en Quick Sort. No obstante, al momento de ser partido el hilo, deduce si el objetivo o índice k -ésimo descansó hacia la izquierda o hacia la derecha. Descartando abruptamente la vía que no posee congruencia asintótica con la ubicación del elemento deseado.

Algorithm 6 Quick Select ▷ Búsqueda indexada selectiva

```

1: procedure QUICKSELECT( $V, low, high, k$ )
2:    $p \leftarrow \text{LOMUTOPARTITION}(V, low, high)$ 
3:   if  $p == k$  then
4:     return  $V[p]$ 
5:   else if  $p > k$  then
6:     return QUICKSELECT( $V, low, p - 1, k$ )
7:   else
8:     return QUICKSELECT( $V, p + 1, high, k$ )

```

Dado que se purga un lado de los datos de forma continua y nunca se ordena el 100 % del conjunto:

- **Mejor y Caso Promedio** $\mathcal{O}(n)$: El sub-arreglo se trunca en proporción constante en la media; procesando $n + n/2 + n/4 \dots$, siendo un orden de magnitud lineal.
- **Peor Caso** $\mathcal{O}(n^2)$: Condicionado igualitariamente por los infortunios teóricos perimetrales de un pivoteo desbalanceado.

4.4. Resultados Experimentales

Esta sección contrasta los hallazgos teóricos con simulaciones obtenidas en los datasets estructurados. El proceso se ejecutó con el *smoke* del binario en modo no interactivo, generando tres conjuntos de datos (ordenado, invertido y aleatorio) de $N = 25000$, encadenando todas las ejecuciones y luego graficando con Python. Esta carga corresponde a la Fase 2 del informe, cuyo objetivo fue revisar el comportamiento empírico de los algoritmos bajo una batería completa de pruebas. Para garantizar una operación sin fallos de cómputo se decidió ejecutar las pruebas de forma secuencial en un único hilo del procesador. La ejecución completa tomó aproximadamente entre 40 minutos y 1 hora.

El estilo de ejecución usado en `smoke.c` fue estrictamente secuencial, encadenando cada experimento y su escritura a CSV antes de iniciar el siguiente. Esta causalidad reduce solapamientos temporales y evita el agotamiento de recursos por concurrencia, garantizando estabilidad y trazabilidad en los resultados.

El entorno de ejecución se resume a continuación; la configuración de hardware y sistema operativo se detalla en el registro técnico adjunto:

1. Sistema operativo: `Linux Mint 22.2 (zara)`
2. Kernel: `6.17.0-23-generic`
3. Arquitectura: `x86_64`
4. CPU: `AMD Ryzen 5 7235HS (4 núcleos / 8 hilos, 1 socket)`
5. Cache L3: `8 MiB`

Las gráficas se presentan en escala lineal; en varios casos las curvas se superponen porque los tiempos reales están en el orden de microsegundos y el redondeo a 10^{-6} s hace que series distintas colapsen visualmente. Esta condición se verifica en los CSV almacenados en el repositorio.

4.4.1. Ordenamiento: Mejor, Peor y Promedio

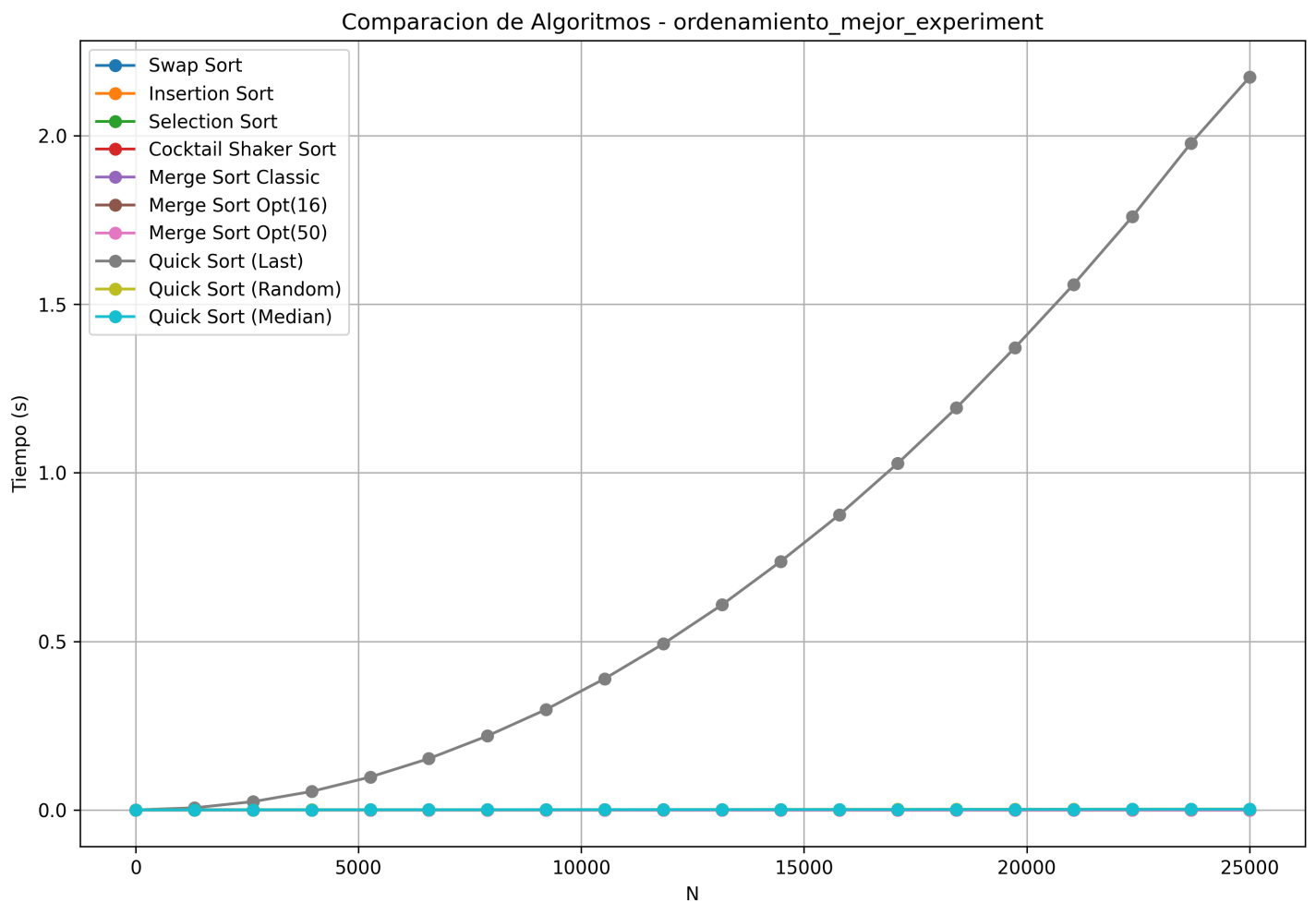


Figura 1. Ordenamiento: mejor caso (arreglo ya ordenado).

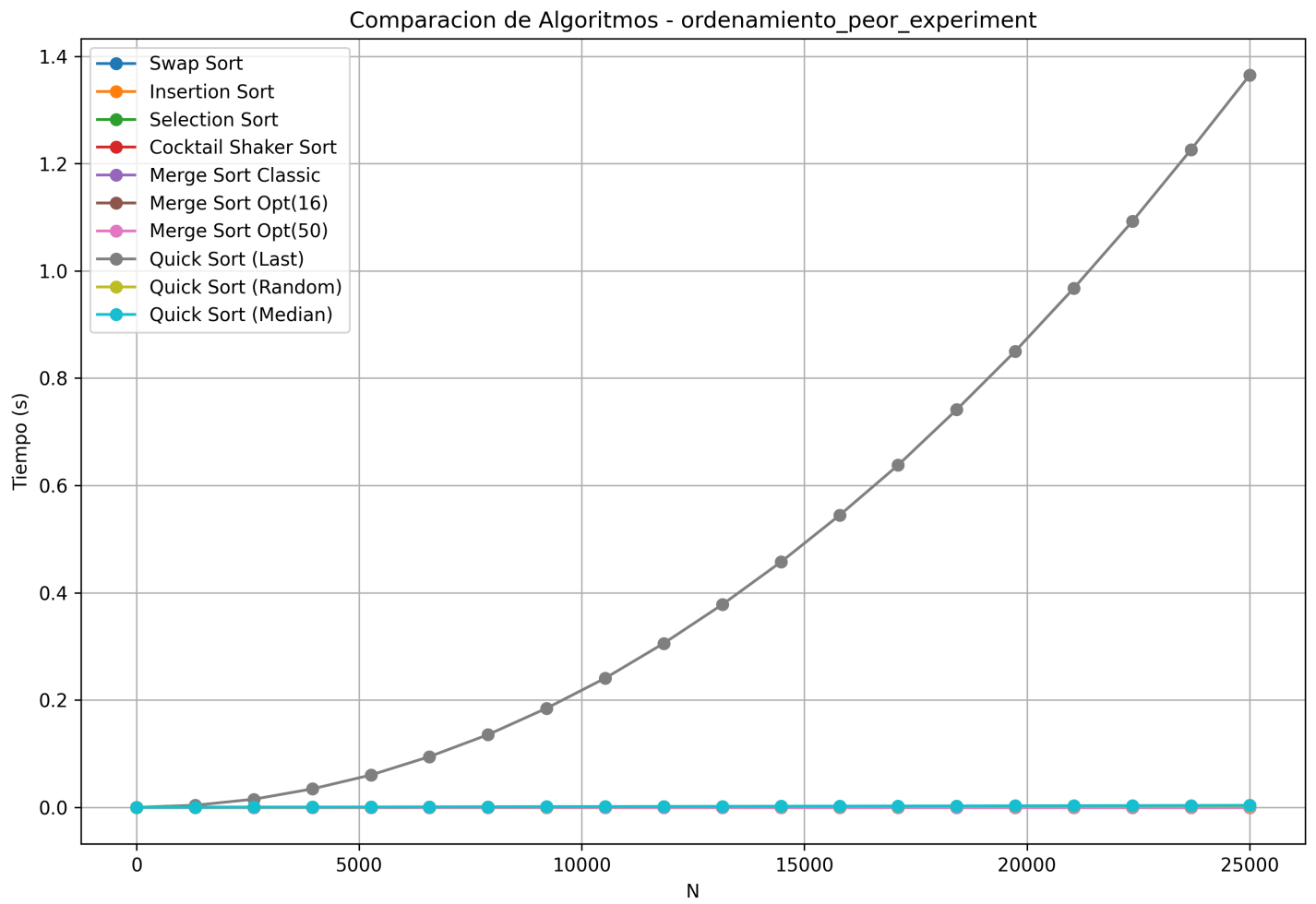


Figura 2. Ordenamiento: peor caso (arreglo invertido).

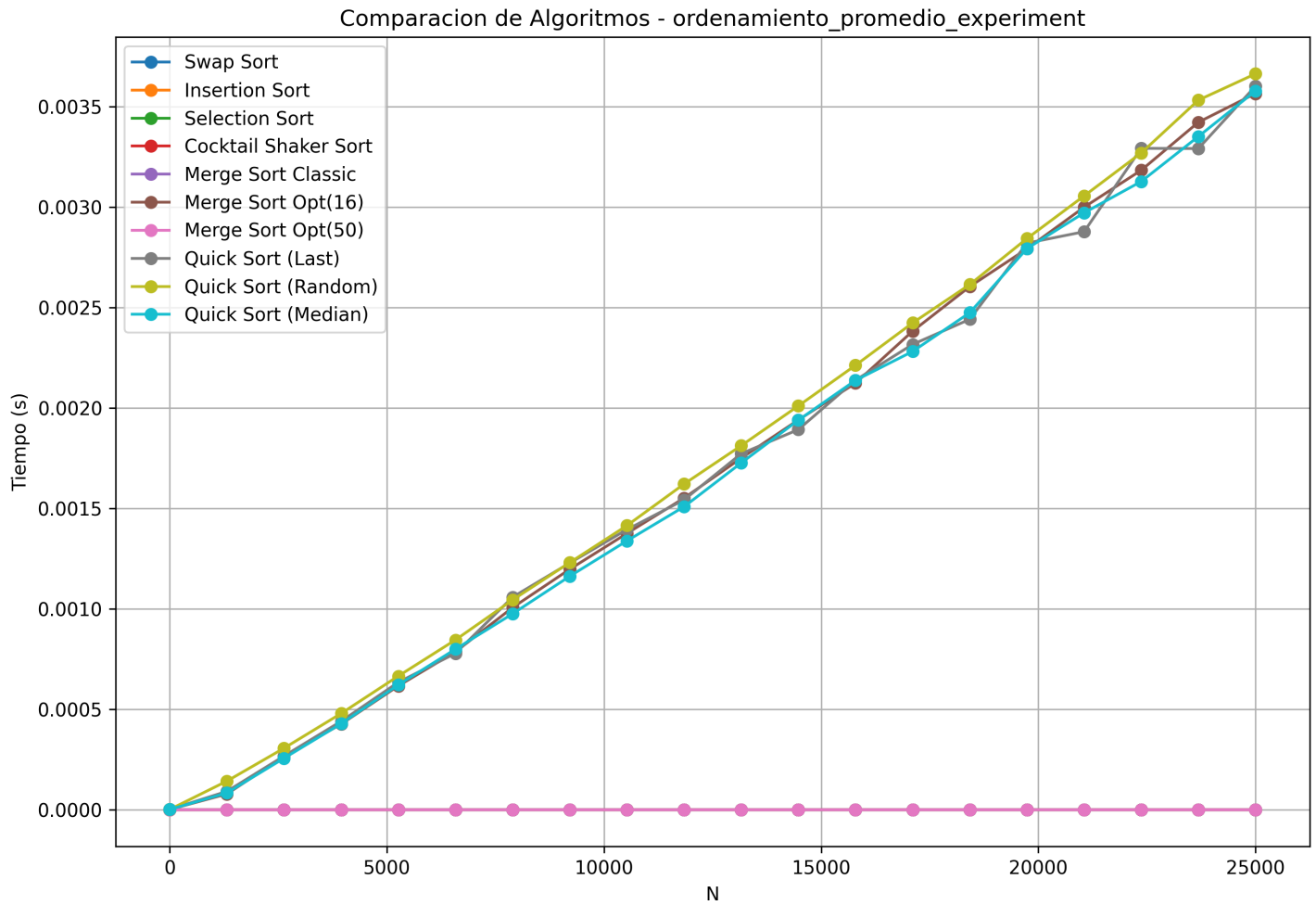


Figura 3. Ordenamiento: caso promedio (arreglo aleatorio).

En los tres escenarios, Quick Sort con pivote aleatorio o mediana se mantiene en el rango bajo de tiempos, mientras que el pivote del ultimo elemento degrada notoriamente en mejor y peor caso (series ya ordenadas o invertidas), exhibiendo el comportamiento esperado de $O(n^2)$. Merge Sort optimizado con umbral 16 sostiene tiempos estables y bajos; el Merge Sort clasico y los algoritmos cuadráticos se acercan a 0.000000 s por efecto del redondeo y la escala, pero sus tendencias se distinguen en el CSV.

4.4.2. Impacto del Umbral en Merge Sort

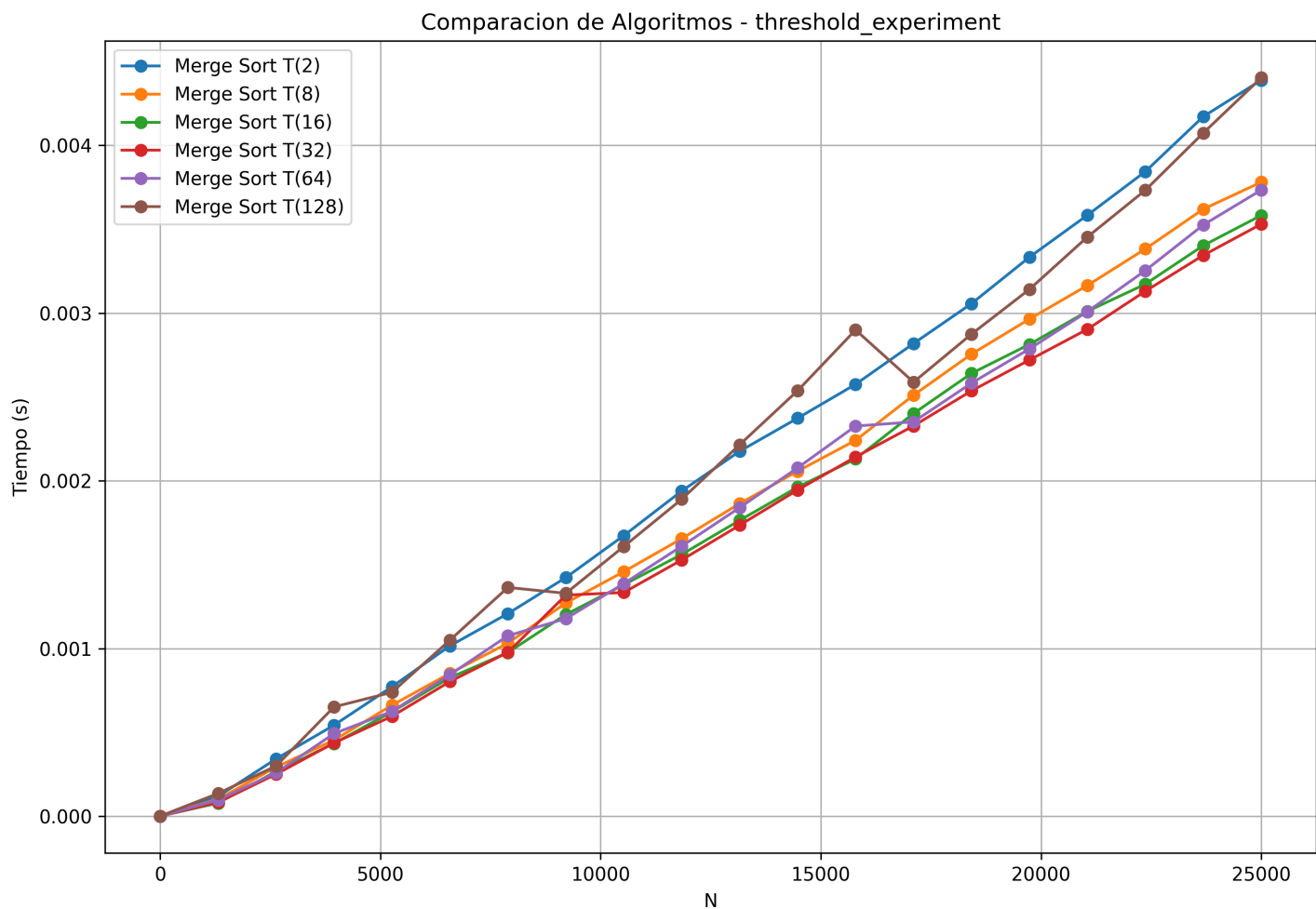


Figura 4. Comparativa de umbrales para Merge Sort optimizado.

La curva muestra que los umbrales intermedios ($T(16)$ y $T(32)$) presentan el mejor equilibrio en todo el rango de N , evitando el costo de recursion excesiva ($T(2)$) y el sobrecosto de ordenar subarreglos grandes con Insertion Sort ($T(128)$). Esto justifica el uso del threshold 16 en las demas pruebas de ordenamiento.

4.4.3. Desempeño en Algoritmos de Búsqueda

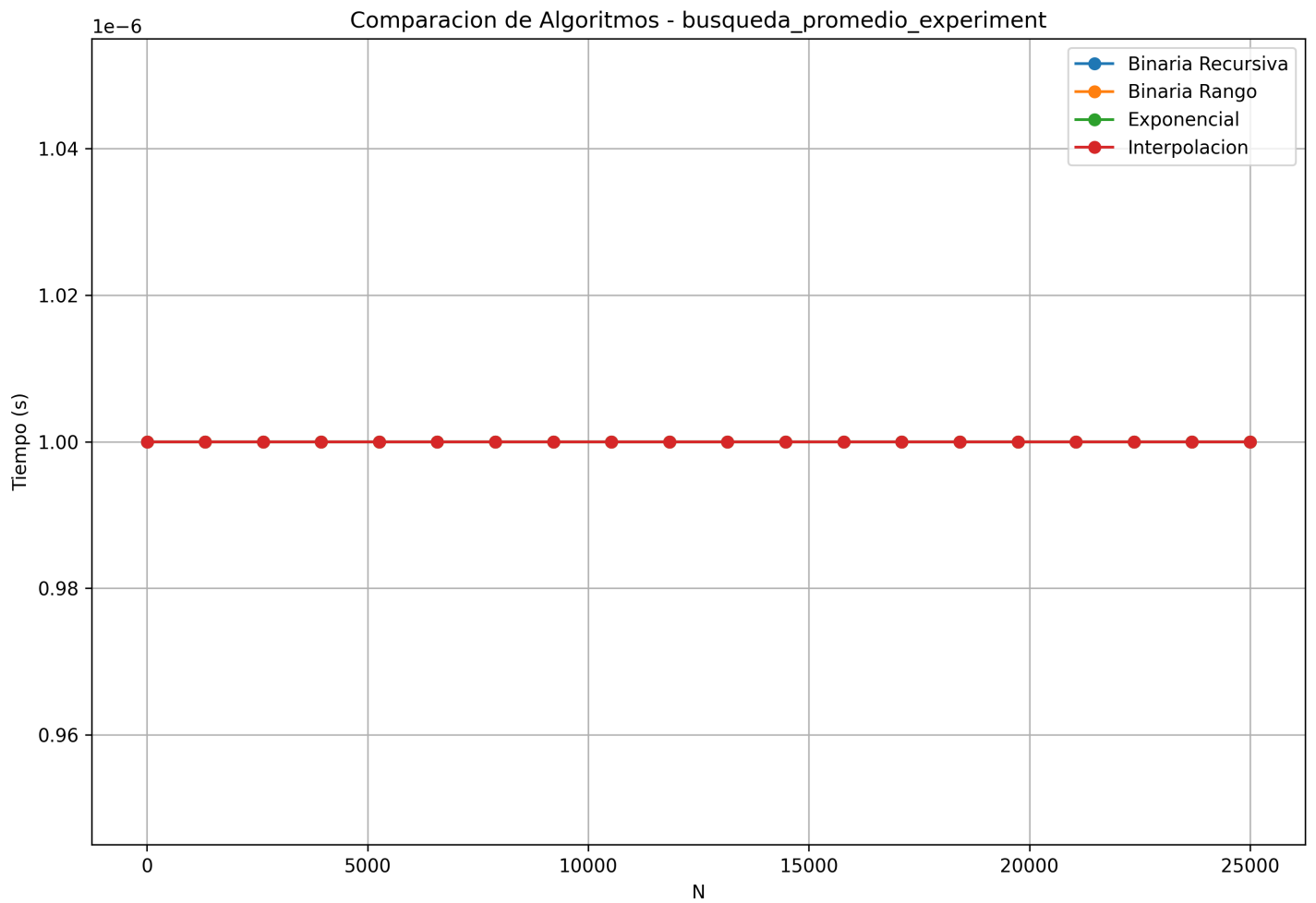


Figura 5. Búsqueda: caso promedio.

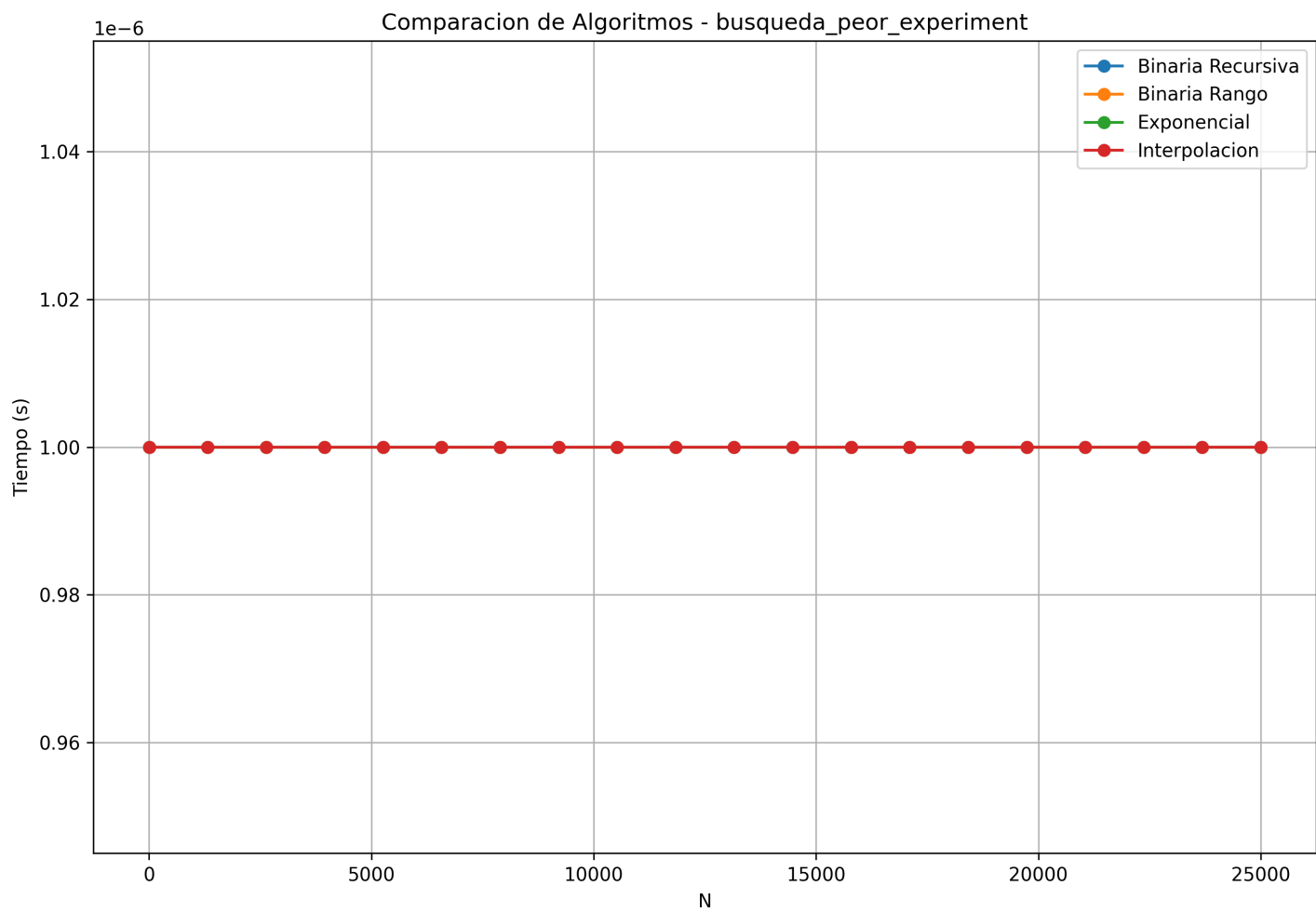


Figura 6. Búsqueda: peor caso.

En ambos casos los tiempos quedan en el orden de 10^{-6} s y se solapan en la grafica. El CSV confirma que todas las variantes se mantienen en valores similares para $N \leq 25000$, lo cual es consistente con complejidades $\mathcal{O}(\log n)$ y un costo constante dominante a este tamaño.

4.4.4. Selección con Quick Select

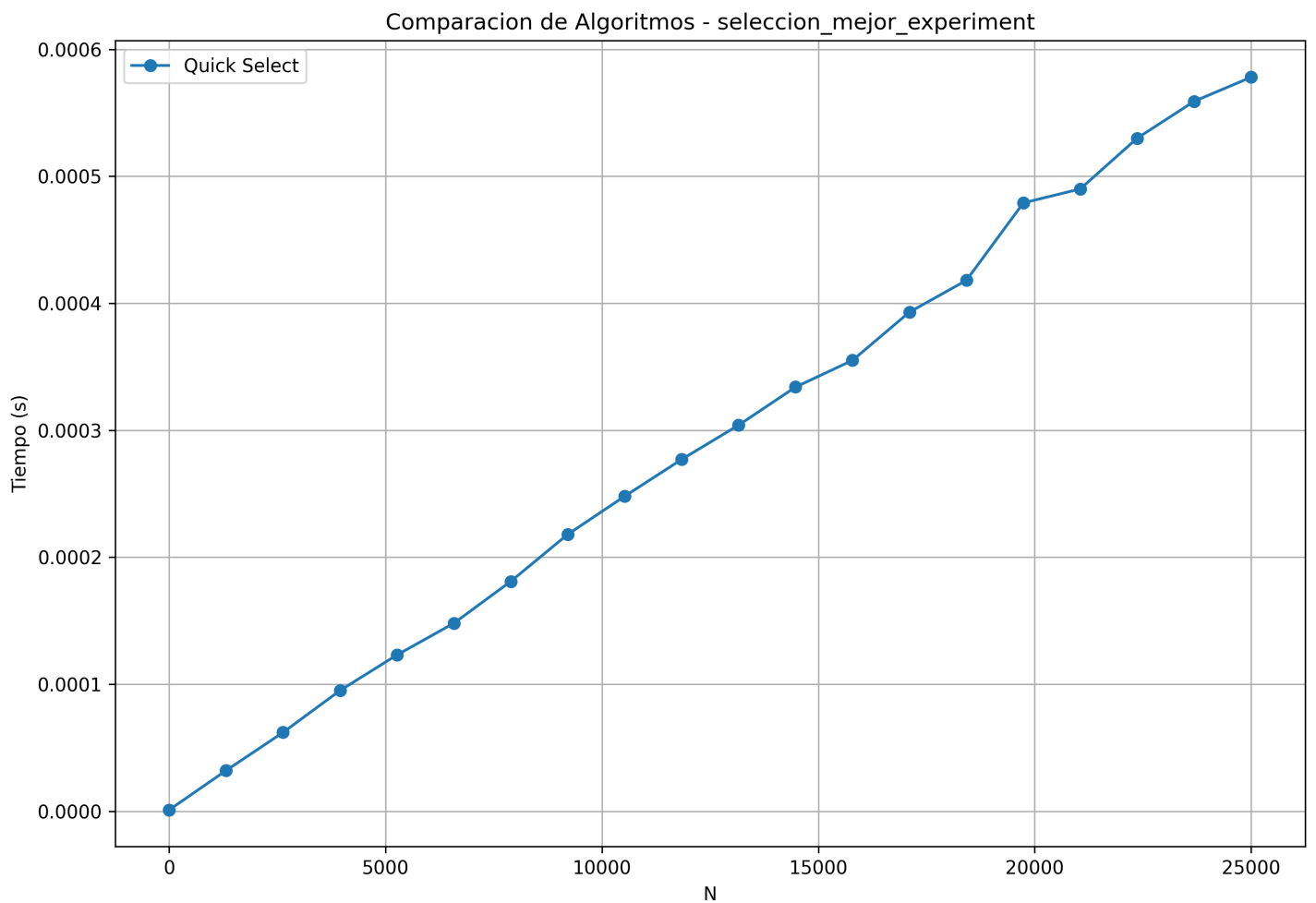


Figura 7. Quick Select: mejor caso (pivote aleatorio en arreglo aleatorio).

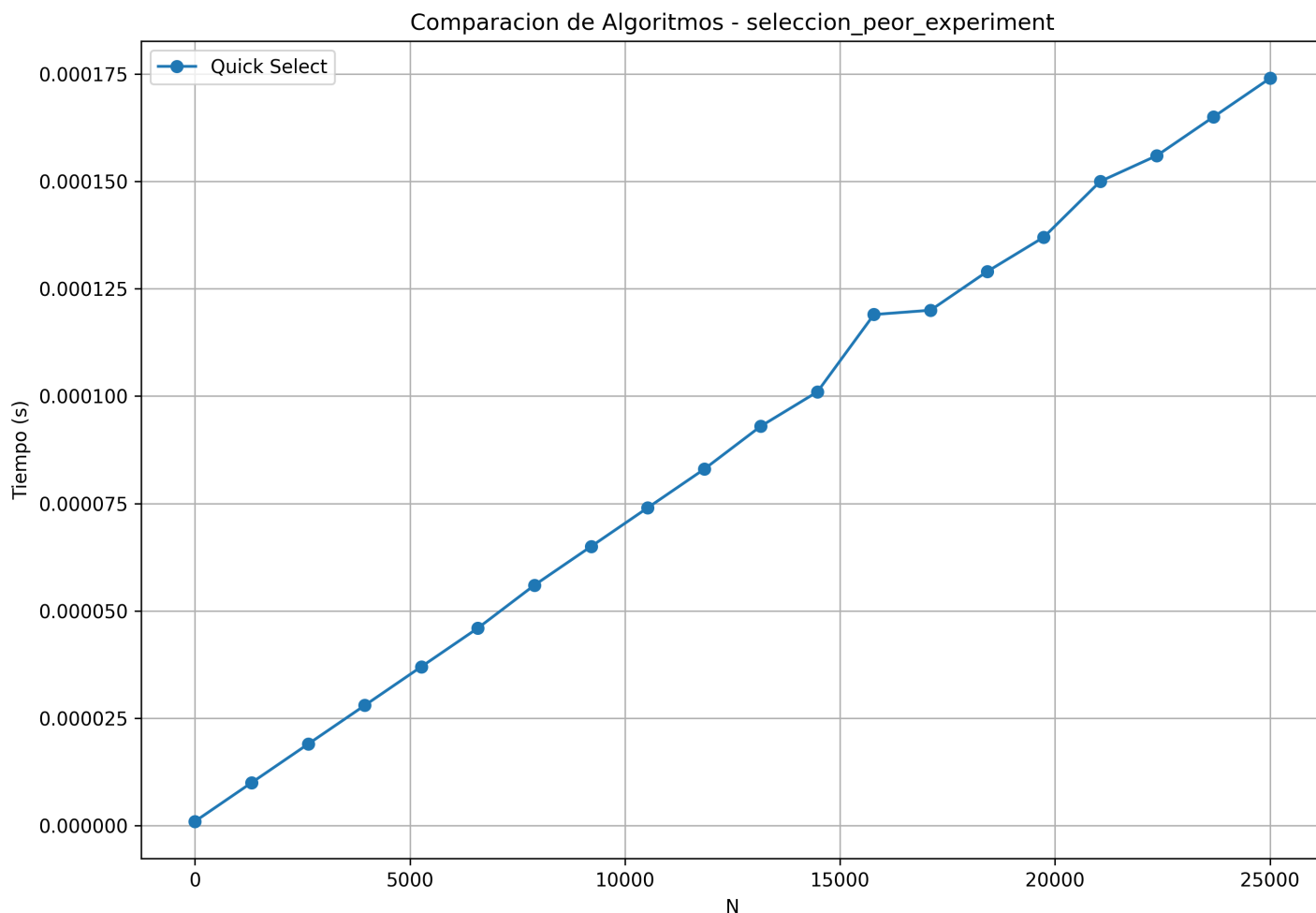


Figura 8. Quick Select: peor caso (pivote ultimo en arreglo ordenado).

Se observa crecimiento aproximadamente lineal en ambos casos, con un peor caso que aumenta más rápido pero mantiene el rango de microsegundos para $N \leq 25000$, coherente con el costo promedio lineal y el peor caso cuadrático amortiguado por el tamaño de entrada y el costo constante de la plataforma.

4.5. Aplicación Práctica y Funcionalidades

El binario interactivo implementado provee un conjunto exhaustivo de herramientas para manipular, analizar y experimentar con estructuras de datos algorítmicas. El menú principal se divide en siete opciones, cada una dirigida a una tarea específica dentro del contexto de análisis de rendimiento de algoritmos de Divide y Vencerás.

4.5.1. Menú Principal y Opciones Generales

El programa arranca con un menú (**MAIN MENU**) que presenta al usuario las siguientes opciones:

1. **Generar nuevo CSV (Generate new CSV):** Permite crear un conjunto de datos de jugadores con tamaño N configurable. El usuario selecciona el tipo de disposición deseada: arreglo ordenado, invertido o aleatorio (shuffled). Internamente, el programa genera estructuras *Player* con identificadores, nombres aleatorios, equipos, puntuaciones y competiciones participadas, almacenándose en formato CSV.

2. **Leer CSV Actual (Read actual CSV):** Carga y exhibe en la consola el archivo CSV existente (`build/db/players.csv`), aplicando paginación automática mediante *more-like* bindings para manejar datasets grandes de forma interactiva.
3. **Ordenar CSV (Sort CSV):** Abre un submenú con 10 opciones de algoritmos de ordenamiento:
 - Algoritmos cuadráticos clásicos: *Swap Sort* ($\mathcal{O}(n^2)$), *Insertion Sort* ($\mathcal{O}(n^2)$), *Selection Sort* ($\mathcal{O}(n^2)$), *Cocktail Shaker Sort* ($\mathcal{O}(n^2)$).
 - Algoritmos divide y vencerás: *Merge Sort Clásico* y *Merge Sort Optimizado con Umbral configurable*, ambos de orden $\mathcal{O}(n \log n)$.
 - Variantes de Quick Sort con distintas heurísticas de pivoteo: último elemento, primer elemento, pivote aleatorio y mediana de tres, todas con complejidad promedio $\mathcal{O}(n \log n)$ pero peor caso variable.

Tras ejecutar el ordenamiento, la consola exhibe el arreglo ordenado de forma paginada.

4. **Búsqueda en CSV (Search value in CSV):** Redirige al submenú **Data Analytics & Rankings**, descrito en detalle más adelante.
5. **Ejecutar Experimentos (Run experiment):** Abre un menú de selección para ejecución de mediciones de rendimiento temporal. Las opciones incluyen: pruebas de umbral para Merge Sort optimizado, experimentos de ordenamiento en tres casos (mejor, peor, promedio), búsquedas (caso promedio y peor), selección, o ejecución de todas las pruebas simultáneamente. Cada experimento genera archivos CSV con tiempos de ejecución.
6. **Ejecutar Smoke + Plots (Run smoke + plots):** Automatiza la ejecución del *smoke test* completo con 25,000 puntos de datos, generando gráficos PNG mediante Python *matplotlib*. Esta es la variante usada en la Fase 2 para las comparaciones experimentales; la Fase 3 reutiliza la misma cadena de validación, pero con 1000 datos de prueba para revisar el funcionamiento estructural del sistema.
7. **Salir (Exit):** Termina la ejecución del programa.

4.5.2. Menú Data Analytics & Rankings

El submenú especializado **Data Analytics & Rankings** (opción 4 del menú principal) implementa cuatro consultas típicas en sistemas de ranking e indexación:

1. **Top N Ranking (por Score):** Ordena el dataset completo de manera descendente por puntuación y expone los N mejores atletas. Internamente, el usuario puede elegir entre **Quick Sort (Mediana de Tres)** con complejidad $\mathcal{O}(n \log n)$ promedio o **Merge Sort Optimizado** con garantía determinística $\mathcal{O}(n \log n)$. La justificación práctica es que requerir el Top N implica necesariamente una fracción continua ordenada; ambos algoritmos, aunque con costos competitivos a nivel de tiempo, proporcionan estabilidad en los datos paginados subsecuentes.
2. **Atleta K-ésimo Mejor (Find K-th Best Athlete):** Localiza el atleta en la posición K del ranking sin necesidad de ordenar la totalidad del dataset. El usuario elige entre dos estrategias:
 - **Quick Select (complejidad promedio $\mathcal{O}(n)$):** Implementa el algoritmo de selección lineal clásico. Mediante particionamiento sucesivo similar a Quick Sort, descarta iterativamente la mitad del espacio de búsqueda que no contiene el K-ésimo elemento, evitando la sobrecarga de ordenamiento completo.
 - **Full Sort + Índice (complejidad $\mathcal{O}(n \log n)$):** Ordena el dataset y accede directamente al índice. Esta alternativa es prácticamente equivalente cuando K es pequeño y la constante de Quick Sort es prohibitiva.
3. **Búsqueda por Rango de Puntajes (Search by Score Range):** Localiza todos los atletas cuyas puntuaciones caen dentro de un rango $[\text{minScore}, \text{maxScore}]$. La implementación ordena previamente el dataset por Score mediante Quick Sort (para agilizar la búsqueda subsecuente) y luego itera de manera lineal a través del arreglo ordenado, recogiendo todos los elementos dentro del rango. Esto aprovecha la proximidad de elementos en el rango ordenado, exhibiendo eficiencia práctica por encima de búsquedas completamente ingenuas.
4. **Búsqueda por ID Exacto (Search Athlete by exact ID):** Localiza un atleta específico utilizando su identificador único. El programa carga el dataset y aplica una búsqueda lineal o bien Búsqueda Binaria si el dataset está previamente ordenado por ID. Esta consulta ejemplifica un caso de uso típico donde la complejidad $\mathcal{O}(\log n)$ binaria supera ampliamente el acceso naive de orden $\mathcal{O}(n)$.

4.5.3. Estructura de Datos y Gestión de Memoria

La entidad central del programa es la estructura **Player**:

Definimos la estructura *Player* de la siguiente manera:

```
struct Player {
    int id;                // Identificador único
    char name[64];         // Nombre del atleta
    char team[64];         // Equipo
    float score;           // Puntuación acumulada
    int competitions;      // Competiciones participadas
    bool potatoe;          // Bandera especial
};
```

Cada registro ocupa aproximadamente 140 bytes en memoria (depende de alineación). Para $N = 25,000$ elementos, la asignación dinámica ronda los 3.5 MB. Los datos CSV se cargan secuencialmente en memoria primaria desde disco, permitiendo operaciones in-place sin transferencias adicionales entre niveles de cache o memoria.

4.5.4. Aplicación de Paradigmas Algorítmicos

A lo largo de todas las opciones del menú, el programa integra efectivamente el paradigma de Divide y Vencerás en contextos prácticos:

- **Escalabilidad Logarítmica:** Las búsquedas binarias, exponenciales e interpolación sobre datos ordenados garantizan tiempo logarítmico, tornando viables consultas sobre millones de registros.
- **Balance Espacio-Tiempo:** Merge Sort sacrifica espacio adicional (necesitando $\mathcal{O}(n)$ de memoria auxiliar) a cambio de garantías determinísticas de tiempo; Quick Sort con Mediana de Tres es más frugal pero con peor caso poco probable en práctica.
- **Combinación de Estrategias:** El menú Data Analytics & Rankings ejemplifica cómo combinar múltiples algoritmos (ordenamiento + búsqueda lineal post-orden, Quick Select sin ordenamiento previo, etc.) según la naturaleza específica de cada consulta.
- **Paginación y Usabilidad:** El programa implementa buffers de salida y paginación (*more-like*) para manejar datasets grandes sin sobrecargar el terminal, manteniendo la claridad visual.

4.5.5. Validación Operativa

Para validar el flujo completo del sistema en un entorno controlado, se ejecutó el *smoke test* con un tamaño reducido de $N = 1000$ elementos, priorizando una verificación funcional rápida. A diferencia de la Fase 2, aquí no se busca volver a evaluar a fondo el rendimiento de los algoritmos, sino comprobar la consistencia estructural del menú, la generación de datos, el encadenamiento de experimentos y la salida de gráficos sobre una carga de prueba menor.

La ejecución generó los CSV de prueba y verificó que el pipeline de generación, lectura y análisis opera correctamente antes de escalar a tamaños mayores. El detalle operacional de esa corrida se resume en los Anexos, donde se conserva el registro de compilación y los tres casos de prueba realizados con 1000 elementos. Los extractos concretos de la compilación (comando `gcc` y advertencias) y los marcadores `=== TEST` aparecen en el Anexo 7.3, y sirven como evidencia de que el binario `build/player.out` fue construido y ejecutado con éxito durante la Fase 3.

5. Conclusiones

A partir del análisis teórico y experimental realizado sobre los algoritmos de la familia *Divide y Vencerás*, y su contraste con los algoritmos iterativos clásicos, se extraen de manera general las siguientes conclusiones:

El salto de cuadrático a log-lineal

Merge Sort y Quick Sort confirman que el cambio de paradigma hacia Divide y Vencerás reduce de forma marcada el costo de ejecución respecto de los algoritmos cuadráticos estudiados previamente. En las pruebas realizadas, las variantes de ordenamiento log-lineal mantuvieron tiempos estables para tamaños altos de entrada, mientras que los métodos de complejidad $\mathcal{O}(n^2)$ se degradaron de manera visible a medida que creció N . Esto valida que la mejora asintótica no es solo teórica, sino que se refleja directamente en la ejecución real.

Optimización empírica: El impacto heurístico

La experiencia empírica mostró que pequeñas decisiones de implementación cambian de forma tangible el rendimiento final. El umbral de Merge Sort permitió aprovechar Insertion Sort en subarreglos pequeños, disminuyendo el costo de la recursión innecesaria. Del mismo modo, Quick Sort con pivote aleatorio o mediana de tres evitó la degradación observada con pivotes deterministas sobre datos ordenados o invertidos. En consecuencia, la heurística elegida influye directamente en la calidad práctica del algoritmo.

Búsquedas avanzadas

Las búsquedas binarias, exponenciales e interpolación muestran que un arreglo previamente ordenado abre la puerta a consultas mucho más eficientes que un barrido secuencial. La búsqueda exponencial resulta útil para acotar rápidamente rangos amplios, mientras que la búsqueda por interpolación ofrece buen comportamiento cuando la distribución de los datos es relativamente uniforme. Sin embargo, su efectividad depende fuertemente del perfil de la entrada, lo que confirma que no todas las optimizaciones son universales.

Decisiones arquitectónicas del uso de memoria

El proyecto también evidenció el costo espacial de cada estrategia. Quick Sort mantiene una ejecución in-place y, por ello, consume menos memoria auxiliar que Merge Sort. En contraste, Merge Sort necesita estructuras temporales para fusionar los subarreglos, a cambio de una mayor estabilidad temporal. La elección entre ambos enfoques depende del equilibrio buscado entre memoria disponible, robustez del tiempo de ejecución y comportamiento frente a distintos tipos de datos.

Cierre de la validación funcional

La Fase 3 complementó la validación experimental con una revisión estructural del sistema mediante pruebas con 1000 elementos, suficiente para confirmar el flujo completo de generación, lectura, ordenamiento, búsqueda y ejecución de experimentos sin repetir la batería pesada de 25,000 datos usada en la Fase 2. Con ello, el informe queda respaldado tanto por evidencia de rendimiento como por comprobaciones de funcionamiento general de la aplicación.

6. Contribución por integrante

A continuación se detalla la contribución realizada por cada uno de los integrantes del grupo en el desarrollo del proyecto.

6.1. Andrés Barbosa

Andrés Barbosa: extendió el código a la solicitud nueva como apoyo técnico y mantenedor original del código heredado.

- Extendió la base heredada para responder a la nueva solicitud, manteniendo compatibilidad con la estructura original del proyecto.
- Brindó apoyo técnico en la integración de ajustes funcionales y en la continuidad del desarrollo del código.
- Conservó criterios de mantenimiento sobre módulos ya validados para evitar regresiones en la implementación.

6.2. Emmanuel Velásquez

El estudiante **Emmanuel Velásquez** contribuyó al proyecto con las siguientes tareas:

- Refactorizó el flujo del programa para incorporar modo desatendido y automatizar la ejecución de pruebas.
- Registró la ejecución de pruebas y dejó trazabilidad de las corridas realizadas durante el proceso de validación.
- Actualizó la documentación del repositorio y adaptó el informe a los requerimientos del Proyecto 2.

6.3. Diego Oyarzo

El estudiante **Diego Oyarzo** contribuyó al proyecto con las siguientes tareas:

- Supervisó las operaciones de ajuste de desarrollo y calidad del informe durante el proceso de cierre.
- Analizó y validó las gráficas emitidas y los archivos CSV generados para verificar la coherencia de los resultados.
- Reforzó la consolidación de fiabilidad del código y su trazabilidad frente a la contraentrega.

7. Anexos

7.1. Registro de validación estructural

La validación de la Fase 3 se ejecutó con un tamaño de prueba de $N = 1000$ elementos para verificar el flujo completo de la interfaz sin repetir la carga pesada de la Fase 2. El registro de ejecución confirma tres casos de generación de datos: arreglo ordenado, arreglo invertido y arreglo mezclado.

- **Compilación:** el binario se generó correctamente con `gcc` y quedó disponible como `build/player.out`.
- **Caso ordenado:** se creó el CSV inicial con 1000 elementos y se comprobó la salida paginada de la tabla de jugadores.
- **Caso invertido:** se repitió la generación con el mismo tamaño para validar que la interfaz acepta disposiciones extremas sin fallos.
- **Caso mezclado:** se verificó la ejecución sobre una distribución aleatoria, confirmando la consistencia del flujo de generación y visualización.
- **Trazabilidad:** el detalle completo de la ejecución quedó registrado en `log.txt`.

7.2. Código base de partición

Cuadro 1. Esquema de partición Lomuto empleado como base en Quick Sort y Quick Select

<code>/**</code>
<code> * @brief Esquema de particion de Lomuto</code>
<code> */</code>
<code>static int lomuto_partition(Player V[], int low, int high,</code>
<code> int pivot_type, int (*comp_f)(Player *, Player *)) {</code>
<code> int p_idx = select_pivot_index(V, low, high,</code>
<code> pivot_type, comp_f);</code>
<code> Player pivot = V[high];</code>
<code> int i = low - 1;</code>
<code> for (int j = low; j < high; j++) {</code>
<code> if (comp_f(&V[j], &pivot) <= 0) {</code>
<code> i++;</code>
<code> swap_player(&V[i], &V[j]);</code>
<code> }</code>
<code> }</code>
<code> swap_player(&V[i + 1], &V[high]);</code>
<code> return i + 1;</code>
<code>}</code>

7.3. Extractos del registro de ejecución

A continuación se incluyen extractos relevantes de ‘log.txt’ que documentan la compilación, advertencias y el desarrollo de la Fase 3 (tests con $N = 1000$). Estos fragmentos soportan la trazabilidad del proceso descrito en la sección de validación operativa.

Cuadro 2. Registro técnico de compilación y ejecución (Fase 3, N=1000)

gcc -Wall -Wextra -Wpedantic -g -c -o obj/generate_exec_times.o src/generate_exec_times.c -I./incs/
src/generate_exec_times.c: In function 'run_experiment':
src/generate_exec_times.c:78:34: warning: passing argument 2 of 'print_error'
discards 'const' qualifier from pointer target type [-Wdiscarded-qualifiers]
78 print_error(101, out_filename, NULL);
gcc -Wall -Wextra -Wpedantic -g -o build/player.out obj/errors.o
obj/generate_exec_times.o obj/generator.o obj/main.o obj/player.o
obj/searching.o obj/smoke.o obj/sorting.o -I./incs/ -lm
◀
=== FASE 3: PRUEBAS DE FUNCIONALIDADES CON 1000 ELEMENTOS ===
Fecha: vie 15 may 2026 10:15:12 -03
◀
=== TEST 1: Generar CSV con 1000 elementos (Sorted) ===
Generating SORTED array...
39.06 KB De memoria reservados
- muestra de salida tabular paginada (IDs 1..25) -
◀
=== TEST 2: Generar CSV con 1000 elementos (Inverse) ===
Generating INVERSE array...
39.06 KB De memoria reservados
- muestra de salida tabular paginada (IDs 1000..976) -
◀
=== TEST 3: Generar CSV con 1000 elementos (Shuffled) ===
- procedimiento análogo al anterior, confirmando consistencia -

■ Referencias

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical algorithms*, 3.^a ed. Addison-Wesley, 1997, ISBN: 978-0-201-89684-8. (visitado 13-11-2019).
- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2.^a ed. Addison-Wesley, 1998, ISBN: 978-0-201-89685-5. (visitado 02-04-2026).
- [3] Linux Kernel Organization, *Linux Programmer's Manual: QSORT(3)*, Accedido el 12 de abril de 2026, 2024. dirección: <https://man7.org/linux/man-pages/man3/qsrt.3.html>.
- [4] A. Barbosa, M. Hernández e I. Gallardo, *PALYER - Proyecto 1 (repositorio original)*, GitHub repository, Fork usado como base para este repositorio, 2025. dirección: <https://github.com/AndrewwhiteCode/PALYER>.
- [5] J. Aldridge Águila, *Diseño de Algoritmos - Tarea 2: Divide y Vencerás (Instrucciones)*, Documento de instrucciones proporcionado por la docente, 2026. dirección: https://pregradovirtual.umag.cl/pluginfile.php/836423/mod_resource/content/10/DAL_Tarea2.pdf.